

# 满减 / 阶梯折扣引擎——平台 ROI 与用户感知的取舍

从五角色视角看一次客诉、一笔补贴、一份运营预算

---

gomall · 业务侧 deck

# 今日大纲

为什么单独抽一个引擎：与优惠券的本质区别

业务困局：满 200 减 25 vs 满 100 减 10，给用户哪一条

规则范围三档：全场 / 类目 / 商品——谁能挂、谁该挂

DailyBudget：运营如何控成本 + 烧完降级

下单 → 关单全链路：apply / release 的事务故事

客服话术：用户说“满了 200 没给我减”怎么回

业务边界：不做什么

运营 / 商家 / 客服路线图（不在本期实现）

压测画像：与现有营销三剑客同坐标系

与 Outbox / Saga 体系的集成

真实事故假想：predictable failure modes

Q&A 与代码位置一览

## 为什么单独抽一个引擎：与优惠券的本质区别

---

## 优惠券和满减是两套独立工具，常被误当同一类

- **优惠券**：用户主动领取 → 落到个人券包 → 下单时用户选。属于”个人资产”。
- **满减**：平台 / 商家挂出的活动规则 → 结算时引擎自动算最优。属于”场域属性”。
- **业务后果差很大**：
  - 优惠券需要面对”羊毛党批量领取”→ Lua 限领 + 风控
  - 满减不需要”领”，但需要面对”运营预算花光”→ 当日 DailyBudget 兜底
- **叠加规则**：本期 gomall 约定**先满减、后券**。引擎只负责前半段；券抵扣由 `internal/coupon/service.go` 在下游算。
- **业务取舍**：一笔订单只能用**一条满减规则**（不叠加多条满减），避免”用户拿三条阶梯都减”的羊毛漏洞。

*internal/promo/model.go:9-13 与 Coupon 的边界注释；internal/coupon/service.go:1-105 不变*

## 5 角色对满减的不同诉求

### C 端用户

”我能减多少？”  
不关心规则细节

### 商家

”我的店搞了满减  
能多带多少单”

### 运营 / 平台

”今天预算 5 万  
烧完该降级吗”

### 客服

”用户问为什么  
我满 200 没减”

### SRE

”calculate  
p99 < 50ms  
apply p99  
< 100ms”

同一套规则表，5 个

角色看到的”成功”定义截然不同。本 deck 每一帧都同步回答这 5 个角色看到了什么。

*internal/promo/model.go:32-46 PromoRule 字段表 + status / DailyBudget 字段服务于不同角色*

## 业务承诺 SLO: calculate 是高频读、apply 是低频写

| 接口           | 业务场景         | QPS 画像 | p99 目标 | 失败时的兜底           |
|--------------|--------------|--------|--------|------------------|
| calculate    | 加购悬浮 / 结算页加载 | 1k-5k  | 50ms   | 退化显示原价 (前端容错)    |
| apply (tx)   | 下单事务内扣预算     | 50-500 | 100ms  | 80003 提示, 订单整体回滚 |
| release (消费) | 关单 / 退款退预算   | 10-100 | 200ms  | Nack 重排重试 + 台账幂等 |
| admin 列表     | 运营后台         | 1-10   | 1s     | 无需 SLO           |

**为什么 calculate 必须更紧:** 用户每次切购物车、改地址都会触发; 如果慢, 整个结算页都被拖。**为什么 apply 可以宽 100ms:** apply 是订单事务的一段, 已经被订单 SLO (< 500ms) 兜住; 自身 100ms 留给 DailyBudget 单 SQL 兜底。[internal/promo/service.go:158-191](#) `CalculateBestDiscount` 的注释里写明 SLO; `README SLO P0/P1` 章节同口径

业务困局：满 200 减 25 vs 满 100 减 10，给用户哪一条

---

## 阶梯门槛真实困局

运营在后台同时配了三条：

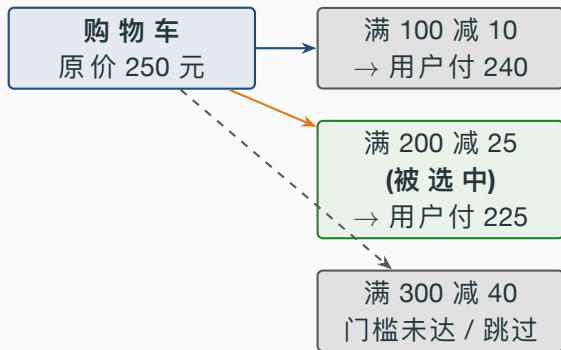
- 满 100 减 10
- 满 200 减 25
- 满 300 减 40

用户购物车 250 元，三条门槛中前两条都满足。给用户哪一条？

- **业内主流：**取“减得最多”的那条——**满 200 减 25。**
- **为什么不是满 100 减 10：**用户预期“我应该拿到更多优惠”，给少了等于“被坑”，舆情风险。
- **平台代价：**补贴成本从 10 变成 25 ——但这是为了**用户感知**支付的钱，运营预算阶段就已经算过。
- **为什么不是满 300 减 40：**用户购物车没到 300，凑不齐就拿不到。这条留给“再加 50 元就能多省 15”的凑单暗示，是**加购引导**的弹药。



## 用户最优 vs 平台最优——一个 70 块钱的取舍



三条规则同时存在的真实业务函数是：

- 满 100 减 10：拉新转化阀（低门槛、低补贴）
- 满 200 减 25：客单价拉升阀（中门槛、中补贴）
- 满 300 减 40：凑单引导阀（高门槛、高补贴，常常拿不到）

三条同时存在 = 三个销售漏斗段同时被覆盖。引擎只是自动选最优那条。

## 满减 vs 满折扣——同价位下哪个更狠

- 同时挂两条都门槛”满 200”：
  - 满 200 减 25
  - 满 200 打 9 折
- 购物车 200 元：减 25 元 vs 减 20 元 ( $200 \times 10\%$ ) → 选**满减**。
- 购物车 500 元：减 25 元 vs 减 50 元 ( $500 \times 10\%$ ) → 选**满折扣**。
- **业务含义**：满折扣是”客单价越高越甜”的杠杆，满减是”刚过门槛就立享”的钩子。两者性质完全不同，运营会有意同时挂。
- **引擎职责**：不替运营做”选择题”，只在”哪条减得多”上做**执行题**。规则之间的合理性由运营负责。

*internal/promo/service.go:108-128 computeDiscountOnBase* 同时覆盖两种类型；

*internal/promo/service\_test.go:49-84* 两条测试分别验证选择方向

## 满折扣的代价：bps 取整带来的“1 分钱”问题

- 折扣 **bps**:  $9000 = 9$  折 (即折后金额是原价的 90%)。
- 算式:  $\text{discount} = \text{base} * (10000 - \text{bps}) / 10000$  (向下取整)
- 极端边界: 购物车 199 元 (19900 分), 9 折 = 减 1990 分 = 19.9 元, 刚好整除。
- 但 **200.01 元** (20001 分) 9 折 = 减 2000 分 = 20 元 (向下取整丢 0.001 分)。  
平台多保 1 分钱, 用户少减 1 分钱。
- 业务上的解释: 永远向下取整保平台。绝不允许向上取, 否则一年累积下来是百万级资损。
- 客服话术口径: 用户算的折扣金额比系统多 1 分钱 → ”系统按整分精度展示, 最终以结算价为准” → 客服可解释。

*internal/promo/service.go:108-128 computeDiscountOnBase* 实现向下取整; *internal/promo/model.go:16-30*

*bps* 常量定义

规则范围三档：全场 / 类目 / 商品——  
谁能挂、谁该挂

---

## 规则范围三档与业务责任分配

| Scope  | 业务含义               | 谁能挂  | 典型预算量级           |
|--------|--------------------|------|------------------|
| 全场 (1) | 平台周年庆 / 618 / 双 11 | 平台运营 | 100 万-1000 万 / 天 |
| 类目 (2) | 图书满 100 / 数码满 500  | 类目运营 | 10 万-100 万 / 天   |
| 商品 (3) | 单 SKU 清仓 / 新品破冰    | 商家自助 | 1 万-10 万 / 天     |

三档对应三个

**不同的钱袋：**平台周年庆烧的是平台预算（市场费用线），类目活动烧的是品类预算（招商运营线），商品级是商家自己补贴自己的货。**合规约束：**金额一律落在 DailyBudget 里走当日预算控制——不允许“无限制让商家烧补贴”，否则容易触发反不正当竞争。

*internal/promo/model.go:20-22 Scope 三档常量；internal/promo/repo.go:77-95 ListActiveForCart 按 scope OR 拼条件*

## 类目级规则：购物车里只有这个类目的金额才算门槛

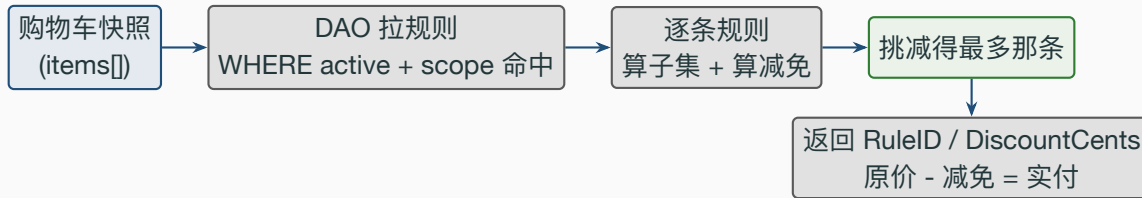
真实场景：购物车 {图书 120 + 数码 30} = 150 元。

- 错的做法：用 **150 元**对照”图书满 100 减 15”→ 用户买 1 本图书 + 1 件 30 元的数码袜子，就能薅”图书”羊毛。
- 对的做法：用 **只算图书的子集合计 120 元**对照门槛 100 → 命中减 15。数码 30 不参与。
- 引擎实现：applicableSubtotal 在每条规则上单独筛子集 → 子集合计 → 门槛对比。
- 业务后果：把”凑单”这件事限制在**品类内**，杜绝跨品类薅羊毛。

*internal/promo/service.go:84-93 applicableSubtotal; internal/promo/service\_test.go:100-119*

*TestPromo\_CategoryScopeOnlyAppliesToCategory* 验证

## 规则匹配流程图



整个 calculate 链路只读一次 DB (ListActiveForCart 一次性把 scope= 全场 OR 命中类目 OR 命中商品全部拉齐), 后面纯内存循环。 *internal/promo/repo.go:77-95 ListActiveForCart;*  
*internal/promo/service.go:158-191 CalculateBestDiscount*

# 核心计算逻辑：30 行 PickBestPromoRule

Listing 1: *internal/promo/service.go:64–82 PickBestPromoRule*

```
64 func PickBestPromoRule(items []CartItem, rules []*PromoRule) (*PromoRule, int64) {
65     var best *PromoRule
66     var bestDiscount int64
67     for _, r := range rules {
68         base := applicableSubtotal(items, r)
69         if base < r.ThresholdCents {
70             continue
71         }
72         d := computeDiscountOnBase(base, r)
73         if d <= 0 {
74             continue
75         }
76         if d > bestDiscount {
77             best = r
78             bestDiscount = d
79         }
80     }
81     return best, bestDiscount
82 }
```



## PickBestPromoRule 逐行讲解

- **第 4 行 for**: 每条规则单独评估, 不剪枝——规则数量级  $< 100$ , 全场循环开销可忽略。
- **第 5 行 applicableSubtotal**: 按 scope 算”作用范围内的金额”。全场 = 总额; 类目 = 子集合; 商品 = 单行小计。
- **第 6–8 行**: 未达门槛直接 continue。门槛是规则的业务硬约束。
- **第 9 行 computeDiscountOnBase**: 在已通过门槛校验的 base 上算减免。
- **第 10–12 行**: 理论上不会出现  $d \leq 0$ , 但配置错误 (如  $bps \geq 10000$ ) 会让  $d=0$ , 跳过。保前端不显示负折扣。
- **第 13–16 行取最大**: 业务取舍——选用户能减得最多的。运营如果想”给少不给多”, 那就别同时挂多条规则。

*internal/promo/service.go:53–82 PickBestPromoRule 的注释里完整写明该业务取舍*

## DailyBudget: 运营如何控成本 + 烧完降级

---

## DailyBudget 是把” 营销预算” 和” 系统兜底” 绑在同一张表里

- **字段**: DailyBudgetCents (当日预算) + ConsumedToday (当日已消耗)
- **业务诉求**: 财务部门每月给营销 100 万预算, 运营拆成 30 天 → 每天 3.3 万 / 规则 → 落到 DailyBudgetCents=3300000。
- **系统兜底**: 超过预算的订单**不再享受满减**, 返回业务码 **80003 PromoBudgetExhausted**。前端展示” 今日满减名额已抢完”。
- **0 = 不限**: DailyBudget=0 表示无上限, 给” 不烧钱” 的规则用 (如 1 折促销但实际门槛 9999 元, 几乎不会命中)。
- **重置**: 每日 0 点 cron 把 ConsumedToday 归零。本期**留路线图**, 引擎本身不依赖 cron 工作。

*internal/promo/model.go:41-42 字段定义; internal/promo/repo.go:97-125 AtomicConsumeBudget 单条*

*UPDATE 带 RowsAffected 兜底*

# AtomicConsumeBudget: 单条 SQL + RowsAffected 兜底

Listing 2: *internal/promo/repo.go:110–125 AtomicConsumeBudget*

```
110 func (d *PromoDao) AtomicConsumeBudget(tx *gorm.DB, ruleID uint, amount int64) error {
111     if amount <= 0 {
112         return nil
113     }
114     res := tx.Model(&PromoRule{}).
115         Where("id = ? AND status = ? AND (daily_budget_cents = 0 OR consumed_today + ? <=
            daily_budget_cents)",
116             ruleID, PromoStatusActive, amount).
117         UpdateColumn("consumed_today", gorm.Expr("consumed_today + ?", amount))
118     if res.Error != nil {
119         return res.Error
120     }
121     if res.RowsAffected == 0 {
122         return ErrPromoBudgetExhausted
123     }
124     return nil
125 }
```

## 为什么是一条 SQL 而不是先 SELECT 再 UPDATE

- **两步法的问题**：A 事务 SELECT 看到 consumed=950，B 事务也看到 950，两边都通过门槛 → 都 UPDATE 成功 → 实际 consumed=1100 > budget=1000，**超发资损**。
- **一条 SQL 把“判断 + 更新”压在 InnoDB 行锁的同一持有期内**，物理上不可能竞态。
- **RowsAffected=0** 是关键信号——不是“DB 出错”，而是“业务条件不成立”。引擎据此转 80003。
- **压测验证**：本地 MySQL 跑 TestPromo\_AtomicConsumeBudget\_Exhaust，三次连续消费 1000 / 1000 / 500 分（预算 1500 分），第二次返回 ErrPromoBudgetExhausted，第三次刚好打满，第四次 1 分都不行——4 个断言全 PASS。
- **真实数字对照**：与 stressTest/REPORT.md 中 Redis Lua 抢券 **500 抢 100 零超发**（max 136ms）思路同源——“判断 + 写入原子化”是高并发资金类操作的通用套路，无论介质是 Redis 还是 InnoDB。

## 运营怎么用 DailyBudget：三档预算策略

### 保守档

10 万 / 天

烧完早，留客诉印象

”今天没抢到，明天再来”

### 标准档

50 万 / 天

匀到下午

16:00 烧完

覆盖下班高峰

### 冲刺档

200 万 / 天

双 11 / 618

不设上限

等同 DailyBudget=0

三档对应不同

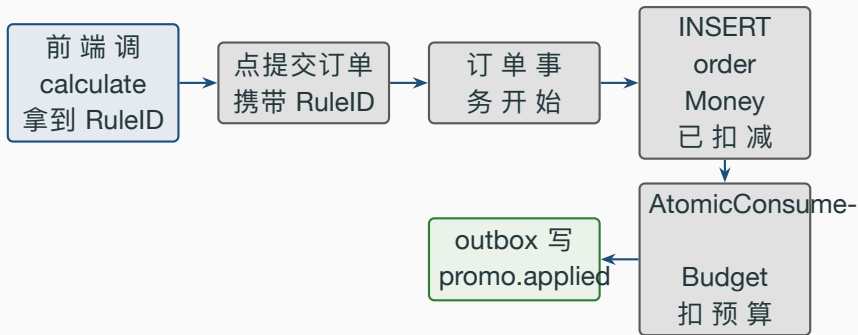
业务节奏：日常活动用保守档稳住成本；大促节点用冲刺档不设限。**烧完不是失败，是设计。**  
`internal/promo/repo.go:121-122 RowsAffected=0` 时返回 `ErrPromoBudgetExhausted`;

`pkg/e/code.go:61-63 80001-80003` 业务码段

下单 → 关单全链路：apply / release  
的事务故事

---

## ApplyDiscountInTx: 嵌在下单事务里



calculate 是“展示”，apply 是“落库”。展示可以反复刷新（高频读），落库只在用户点提交时一次（低频写）。二者不共享缓存，避免“看到 25 元、实际只减了 15 元”的不一致投诉。 [internal/promo/service.go:225-249](#)  
[ApplyDiscountInTx](#) 嵌入业务事务； [internal/order/service.go:137](#) 同模式调 `outbox.Insert`



# ApplyDiscountInTx: 14 行的事务嵌入

Listing 3: *internal/promo/service.go:234–249 ApplyDiscountInTx*

```
234 func (s *PromoSrv) ApplyDiscountInTx(tx *gorm.DB, orderID, ruleID uint, discountCents int64)
    error {
235     if ruleID == 0 || discountCents <= 0 {
236         return nil
237     }
238     if err := NewPromoDaoByDB(tx).AtomicConsumeBudget(tx, ruleID, discountCents); err != nil {
239         return err
240     }
241     return outbox.NewOutboxDaoByDB(tx).Insert(
242         "promo", "PromoApplied", "promo.applied", orderID,
243         events.PromoApplied{
244             OrderID: orderID,
245             RuleID: ruleID,
246             DiscountCents: discountCents,
247         },
248     )
249 }
```

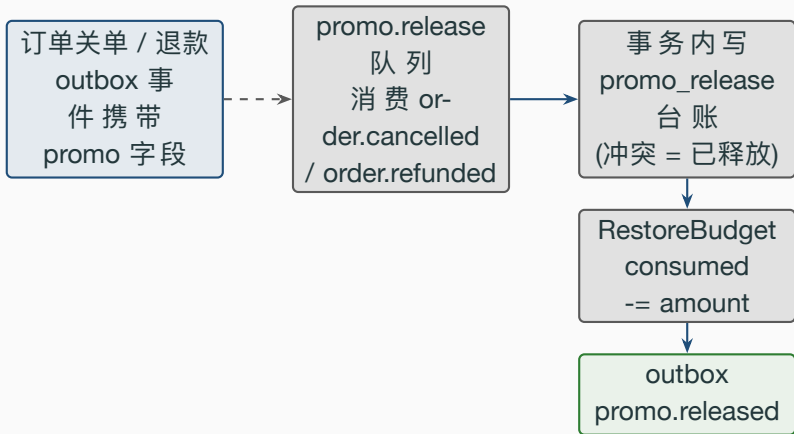
- **第 1 行签名 tx gorm.DB**: 调用方持有事务句柄。这是 outbox 模式的标准入口——业务写 + outbox 写要在同一事务。
- **第 2-4 行守门**: ruleID=0 (没适用规则) 或 discountCents=0 (理论不应发生) 时直接 nil, 不写预算、不写 outbox。
- **第 5-7 行扣预算**: 失败的可能性有两种——DB 错误 (5xx) 或预算耗尽 (80003)。两者都向上 return, 整个订单事务回滚。
- **第 8-14 行写 outbox**: 事件类型 PromoApplied, 下游订阅者: 财务系统 (统计补贴成本)、商家看板 (活动效果)、风控 (异常高频命中)。
- **为什么写 outbox 不是直接发 MQ**: 与订单事务原子。MQ 失败、outbox 写成功 = 后台 publisher 重试; outbox 写失败、订单事务失败 = 整个回滚——双写问题被天然消解。

*internal/promo/service.go:234-249; internal/shared/outbox/publisher.go:62-84 publisher 异步发布 outbox 行*

## ReleaseDiscount: 消费关单 / 退款事件, 异步退还预算

- **时机**: order 侧不再直调——`promo.release` 队列消费 `order.cancelled / order.refunded` 事件后触发。事件载荷自带 `promo_rule_id / promo_discount_cents`, 消费侧不反查订单表。
- **幂等**: 事件是 at-least-once 投递, 同一订单可能重复进来。事务内先写 **promo\_release 台账** (`order_id` 唯一索引 + `ON CONFLICT DO NOTHING`), `INSERT` 冲突 = 已释放过, 直接返回——不重复回补预算、不重复发事件。
- **业务约束**: 不要求规则仍在 **active** ——即便运营已经把规则停用了, 已发生的订单退款仍要退还预算, 否则 `ConsumedToday` 会错算。
- **兜底**: `WHERE consumed_today >= ?` ——不允许减成负数。即便 `ConsumedToday` 已被 cron 重置归零, `RowsAffected=0` 也不返回错误 (视为“今天的预算已经过期, 无需归还”)。
- **outbox 事件**: `promo.released` 携带 `reason = cancel / refund / manual`, 下游统计补贴回收来源。
- **业务诚实**: **cancel** 退还百分百 (用户从未享受过补贴), **refund** 全单退则全退

## Release 链路示意



为什么 **release** 不嵌入关单事务：事件载荷自包含，promo 侧异步消费——退预算失败只 Nack 重排自己，不影响关单本身；毒消息（解码失败）不重排，交给 outbox 重发 / 死信兜底。重复投递由 promo\_release 台账幂等吸收22/37  
最终一致 internal/promo/service.go:261, 284: internal/promo/consumer.go:62, 84 StartReleaseConsumer:

客服话术：用户说”满了 200 没给我  
减” 怎么回

---

## 客户问”为什么我满了 200 没减 25”——四种可能

| 业务码   | 真因          | 客服话术                                   |
|-------|-------------|----------------------------------------|
| 80001 | 规则已过期 / 未开始 | ”您看到的活动是 X 月 X 日生效，今天活动已结束。”           |
| 80002 | 不在适用范围      | ”这条满减是图书类目的，您购物车里图书只有 80 元，未达 100 门槛。” |
| 80003 | 预算用尽        | ”今天该满减活动名额已抢完，欢迎明天 0 点再来。”             |
| N/A   | 您还没到 200    | ”您当前购物车 198 元，再加 2 元就能享受满 200 减 25。”   |

业务码 80001/80002/80003 与现有 70001/70002/60002（限流 / 熔断 / 幂等）口径一致。客服 SOP 培训只需要拿这张表 + 复述句式。 [pkg/e/code.go:61-63](#) 业务码 80001-80003； [pkg/e/msg.go:54-56](#) 客服话术原句

Listing 4: *pkg/e/code.go*:61–63 + *pkg/e/msg.go*:54–56 业务码 + 客服话术同步

```
61 // pkg/e/code.go
62 PromoRuleExpired = 80001 // 规则已过期 / 未生效
63 PromoRuleNotApplicable = 80002 // 购物车未达门槛 / 范围不匹配
64 PromoBudgetExhausted = 80003 // 平台当日预算用尽
65
66 // pkg/e/msg.go
67 PromoRuleExpired: "满减活动已结束或未开始",
68 PromoRuleNotApplicable: "当前购物车未满足满减门槛或不在适用范围",
69 PromoBudgetExhausted: "本场满减活动当日预算已用完, 欢迎下次再来",
```

## 客服真实事故脚本（演练材料）

- 用户：“我刚才看着是减 25 啊，怎么下了单只减了 10？”
- 客服第 1 步：让用户把订单号给我，traceld 贴出来。
- 客服第 2 步：ELK 检索 traceld，找到 `promo.GetPromoSrv().CalculateBestDiscount` 的入参 / 出参；
- 客服第 3 步：对比 calculate 出参（RuleID=2，25 元）与 order 表 Money 字段（实付差 10 元）。
- 常见真因：用户在 calculate 展示后修改了购物车（删了一行商品），重新提交时引擎按新购物车算了不同的 RuleID。这不是 bug，是预期。
- 客服话术：“您结算前购物车有调整，引擎按最新的购物车重新算了最优满减。如果想保留 25 元那条，请确保购物车不要低于 200。”
- 改进路线图：calculate 返回时缓存一个 promo\_token，下单提交带回；引擎校验 token + 购物车哈希一致才落该 RuleID。本期未做。

*internal/promo/service.go:158-191 calculate 是无状态读；internal/promo/service.go:234-249 apply 重新算 = 用25/37*



业务边界：不做什么

---

## 不做清单——诚实标出本引擎的边界

- 不做用户领券：那是 `internal/coupon/service.go` 的职责。两套系统通过下单链路串起来。
- 不做跨店满减：本期 `PromoRule.BossID` 字段都没有 → 类目 / 商品级实际只对单店有效。”满 X 减 Y 任意店”留路线图。
- 不做支付通道折扣（如”支付宝立减 5”）：那是支付通道侧的事，gomall 不与第三方支付通道议价。
- 不做会员价：本期没有 VIP 等级表。Plus / 黑金会员价单独建模，不挂在 `promo` 里（避免规则爆炸）。
- 不做规则编辑历史：admin 改了 `DiscountCents` 直接覆盖，没有 `audit log`。真实事故场景下需要补 `audit_log` 表追责，路线图。
- 不做 A/B 测试位：哪条规则在哪个流量桶展示，留给前端 A/B 框架。引擎只回答”这个购物车最优是哪条”。

运营 / 商家 / 客服路线图（不在本期实现）

---

## 本期落地：结算集成 + 关单 / 退款 release 全闭环

- **[已落地]** 同步下单接入满减：internal/order/service.go::OrderCreate 已调用 CalculateBestDiscount + ApplyDiscountInTx，订单写入 PromoRuleID / PromoDiscountCents / FinalCents 三字段。预算耗尽走降级 (PromoRuleID=0，不阻断下单)。order 侧通过构造器注入的 PromoCalculator 接口调用，不再硬编码 GetPromoSrv。
- **[已落地]** 关单退预算（异步）：CancelUnpaidOrder 不再直调——order.cancelled 事件载荷携带 promo 字段，promo.release 消费者调 ReleaseDiscount(reason=cancel)。
- **[已落地]** 退款退预算（异步）：ApproveRefund 同理——order.refunded 事件驱动 ReleaseDiscount(reason=refund)，重复投递由 promo\_release 台账幂等吸收。
- **[已落地]** 测试：下单侧 4 例（命中 / 不命中 / 多规则取最优 / 预算耗尽降级，TestOrderCreate\_\*）+ release 侧 4 例（台账幂等 / 多订单独立 / 事件解析 / 退款释放，TestPromo\_Release\*）全 PASS。

## 满减引擎的下一步：补 admin UI 与商家自助

- 已落地：规则 CRUD、calculate / apply / release、80001-80003 业务码、**下单链路 + 关单 / 退款 release 全闭环**。
- 路线图 P1：异步下单链路 OrderEnqueue 接满减（与同步 OrderCreate 同口径调 CalculateBestDiscount + ApplyDiscountInTx）。
- 路线图 P1：商家自助挂**商品级**满减（Scope=3 + BossID 校验），与 merchant.Group 一起做。
- 路线图 P1：admin UI 看板——每条规则今日补贴金额 / 命中订单数 / ROI。数据已经在 **outbox** 里，离线 ETL 跑就行。
- 路线图 P2：DailyBudget 0 点重置 cron + 重置事件 promo.budget\_reset，与 initialize/cron.go 一致风格。
- 路线图 P2：promo\_token ——calculate 时颁发，apply 校验，杜绝“用户改购物车导致前后不一致”。
- 路线图 P3：与 Coupon 的叠加规则在 internal/order/service.go 算式层固化，单测覆盖“先满减、后券”5-10 个 case。

## 压测画像：与现有营销三剑客同坐标系

---

## 与现有营销三剑客的 RPS 对照

| 接口                            | RPS            | p95                  | 备注                   |
|-------------------------------|----------------|----------------------|----------------------|
| 优惠券领取 (Redis Lua)             | 51,362         | 3.52ms               | PR #50 实测            |
| 秒杀 + 滑动窗口                     | 52,082         | 1.24ms               | PR #49 实测, 30VU 通过 4 |
| 幂等下单 (同 key 50VU 15s)         | 50,319         | 2.33ms               | 755,033 请求 → 1 笔订单   |
| <b>promo.calculate</b> (路线图)  | <b>目标 50k+</b> | <b>目标 &lt; 50ms</b>  | 一次 DB + 内存循环         |
| <b>promo.apply (tx)</b> (路线图) | <b>目标 10k+</b> | <b>目标 &lt; 100ms</b> | 受订单事务 SLO 兜底         |

calculate 由于只读、单 DB 查询, RPS 应当接近秒杀 / 优惠券档位 50k+。apply 受订单事务

SLO < 500ms 整体兜住, 自身目标 100ms。两个目标都在 stressTest 数据规模 (order 6M 行 / product 956K 行) 下可达。stressTest/REPORT.md ”各链路压测结果”; README 真实压测数据章节

## 真实数字之二：DAO 单测的并发安全验证

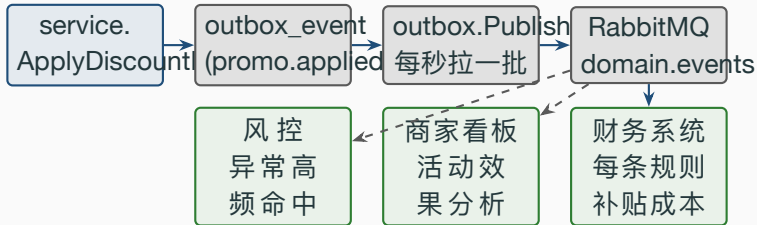
- 测试：  
`internal/promo/repo_test.go::TestPromo_AtomicConsumeBudget_Exhaust`
- 场景：DailyBudget=1500 分，连续 4 次 AtomicConsumeBudget(1000, 1000, 500, 1) 应当 PASS / FAIL / PASS / FAIL
- 实测：4 个断言全 PASS，耗时 **0.29s**（含 MySQL InnoDB 事务开销）
- 对照：单条 UPDATE + RowsAffected 兜底 = 与 stressTest/REPORT.md 中 Redis Lua 抢券”500 抢 100 零超发”同源思路。
- 差异：Lua max 136ms vs InnoDB 单 UPDATE 约 1-5ms（轻负载下 InnoDB 反而更快），高竞争下 Lua 大幅领先。当前满减并发量级（500-1000 单/s）远低于秒杀级别，InnoDB 即可。
- 压测路线图：补 promo\_calculate.js / promo\_apply.js 两份 k6 脚本，500VU × 30s 验证 RPS 与 RowsAffected=0 出现频次。



## 与 Outbox / Saga 体系的集成

---

## promo 在 outbox 体系里的位置



一条 `promo.applied` 事件喂三个下游消费者。三者订阅同一 routing key，互不影响——这就是 outbox 模式的“事件单源 + 多订阅”红利。 *internal/shared/outbox/publisher.go:62-84 publisher 主循环*；  
*service/events/events.go:137-152 PromoApplied / PromoReleased payload*

## 为什么不直接调下游 HTTP / RPC：双写问题

- **反例：** apply 成功后直接 `http.Post(finance, payload)` → 财务系统挂了 5 秒，下单事务也被卡 5 秒 → 用户看 5 秒转圈 → 放弃。
- **反例 #2：** 先写 DB 再发 HTTP，DB 成功 / HTTP 失败 = 财务漏一条补贴，月底对账时凭空多出 25 元缺口。
- **outbox 解法：** 写 outbox 是 DB 同一事务，要么一起成、要么一起回滚。后台 publisher 异步发 MQ，失败重试，至少一次语义 + 下游必须幂等。
- **对应 deck 11：** "Outbox 与一致性" 对此有完整论证，本 deck 不重复——只指明 `promo.applied / promo.released` 沿用了同一套基础设施。

*docs/slides/11-consistency.tex* "双写问题" 章节；*internal/promo/service.go:241,284* `outbox.Insert` 调用

**真实事故假想：predictable failure  
modes**

---

## 假想故障 1：客服收到大量“我满了没减”投诉

- **症状：**大促当天 10 点，客服 1 小时内收到 200+ 条“满 200 没减 25”投诉。
- **排查 5 步：**
  1. ELK 检索 `traceId + "promo.applied"` ——看实际 apply 了哪条规则
  2. 检查 `promo_rule` 表 `status=2 (stopped)` 数量——是否运营误关
  3. 检查 `consumed_today` 是否已  $\geq$  `daily_budget_cents` ——是否预算耗尽
  4. 检查 `calculate` 入参 vs `apply` 入参的购物车快照 `hash` ——是否用户改了购物车
  5. 检查 `promo.applied` 事件量 vs `promo.released` 事件量——是否退款风暴
- **大概率真因：**DailyBudget 设小了——运营在配置后台填 1500000 (1.5 万元) 当 15000000 (15 万元) 写少一个 0。
- **修复 SOP：**POST `/admin/promo/rules` 重新挂一条正确预算的规则，再 PATCH `/admin/promo/rules/:id/stop` 停掉错配那条（两个 admin 接口都已就位），活动近乎无缝衔接。

*internal/promo/repo.go:141-145 Stop 接口； internal/promo/handler.go:62-94 AdminCreate / AdminStop handler*

## 假想故障 2：财务对账每月差几百块

- **症状：**财务每月跑对账，promo 补贴明细总比 DB ConsumedToday 累计少 200–500 元。
- **常见真因：**cron 每日 0 点 reset 之前的最后一秒，有订单进来 apply → ConsumedToday +25 → 0 点重置归零 → 这笔补贴没有被任何“日报”记录。
- **解法：**财务对账以 **outbox** 事件为准，不以 ConsumedToday 为准。  
ConsumedToday 是当日限额执行用，不是对账依据。
- **为什么 outbox 是金本位：**每条 promo.applied 都是一条不可篡改的事件，order\_id / rule\_id / discount\_cents 全有。月底跑 SUM(discount\_cents) WHERE rule\_id=X AND created\_at BETWEEN ... 即可。
- **业务诚实：**ConsumedToday 是运营视角的“今天用了多少”，outbox 是财务视角的“我们补贴了多少”。两个视角不强一致——这是设计，不是 bug。

*internal/promo/service.go:234–249 写 outbox；internal/shared/outbox/publisher.go:62–84 至少一次语义*

## Q&A 与代码位置一览

---

## Q&A（上）——业务侧高频问题

- **Q1**: 满减能不能与优惠券叠加? **A**: 能。规则是先满减、后券, 由 `internal/order/service.go` 算式层固化。本期引擎只算前半段。
- **Q2**: 一笔订单能不能用两条满减? **A**: 不能。`PickBestPromoRule` 只返回一条——防止多条阶梯被同时享受。
- **Q3**: 用户加购到 199 元, 前端要不要主动提示”再加 1 元享受满 200 减 25”?  
**A**: 要。前端可以调 `calculate` 拿到**最接近的下一档规则** (本期接口未直接返回, 路线图)。
- **Q4**: 商家能不能给**特定用户**做满减? **A**: 不能。引擎当前没有”用户白名单”维度。如果商家想做 VIP 专享满减, 应该走优惠券 (VIP 自动发券)。
- **Q5**: 规则一旦激活, 能改 `Threshold / Discount` 吗? **A**: 可以改, 但**不影响已 apply 的订单**。已 apply 的金额已经写进 `outbox` 和 `order` 表。改后只影响新单。



## Q&A (下) ——技术侧高频问题

- **Q6:** 为什么金额一律用 int64 分? **A:** 浮点累计误差。1.10 + 1.20 在 float64 下不等于 2.30。订单 / 财务 / 退款全链路用分, 杜绝精度问题。
- **Q7:** calculate 不签 token, 下单时购物车变了怎么办? **A:** 当前是 apply 时重新算一次, 可能选到不同的 RuleID。客服话术已经覆盖该场景。promo\_token 路线图见前文。
- **Q8:** DailyBudget 用尽后用户能不能看到” 今日 0 点再来”? **A:** 业务码 80003 + msg.go 已写” 本场满减活动当日预算已用完, 欢迎下次再来”。前端拿业务码翻译即可。
- **Q9:** promo.applied 事件丢了怎么办? **A:** outbox publisher 有指数退避重试, 超 MaxAttempts=10 次标记 dead 后报警。运营手动补单。
- **Q10:** 规则量级到多少时需要加索引? **A:** 当前 scope / scope\_ref\_id / status / start\_at / end\_at 都有索引。规则总数 > 10k 时考虑 (status, scope, scope\_ref\_id) 复合索引。

# 代码位置一览

| 模块                                       | 位置                                                     |
|------------------------------------------|--------------------------------------------------------|
| 模型 (PromoRule + 常量)                      | internal/promo/model.go                                |
| DAO (CRUD + 预算扣减 / 退还)                   | internal/promo/repo.go                                 |
| DAO 单测 (预算耗尽 / 退还 / 范围查询)                | internal/promo/repo_test.go                            |
| Service (最优算法 + 事务嵌入 + outbox)           | internal/promo/service.go                              |
| Service 单测 (阶梯 / 满折扣 vs 满减 / 范围)         | internal/promo/service_test.go                         |
| Release 消费者 (order.cancelled / refunded) | internal/promo/consumer.go (启动: initialize/promo.go)   |
| Release 测试 (台账幂等 / 事件解析)                 | internal/promo/release_test.go                         |
| Service 事件 payload                       | service/events/events.go::PromoApplied / PromoReleased |
| API handler (公开 + admin)                 | internal/promo/handler.go                              |
| 路由注册 (公开 / authed / admin 三组)            | internal/promo/routes.go                               |
| 业务码与提示语                                  | pkg/e/code.go:61--63 + pkg/e/msg.go:54--56             |
| Migration                                | internal/migrate/migrate.go                            |
| DTO                                      | internal/promo/dto.go                                  |

引擎自身无新增第三方依赖, outbox / 业务码 / RMQ 消费体系完全复用既有基础设施。